

Composable Correctness and Progress Guarantees

Document #: P4214R0 [[Latest](#)] [[Status](#)]
Date: 2026-05-10
Project: Programming Language C++
Audience: SG1
Reply-to: Lucian Radu Teodorescu (Garmin)
<lucteo@lucteo.ro>

Contents

- [1 Introduction](#)
 - [1.1 Lamport’s safety and liveness](#)
 - [1.2 Safety composability](#)
 - [1.3 Liveness composability](#)
- [2 A pivot on liveness](#)
- [3 Correctness composability](#)
 - [3.1 An example](#)
 - [3.2 Discussion](#)
 - [3.2.1 Undefined behavior](#)
 - [3.2.2 On the difficulty of composing safety](#)
 - [3.2.3 On the difficulty of composing progress](#)
 - [3.2.4 Working with uncertainty](#)
- [4 Takeaways](#)
- [5 References](#)

§ Abstract

Correctness is usually harder to discuss than the individual properties from which it is built. Following Lamport’s division of correctness into safety and liveness [[Lamport77](#)], this paper argues that C++ standardization should treat progress guarantees as part of the correctness contract of concurrency facilities.

Safety properties are local and compose naturally when component assumptions are preserved: if no part of a program violates its invariants, composing those parts does not by itself create a safety violation. Liveness properties do not generally compose, because progress is often global, behavioral, and scheduler-dependent. However, once safety is established, the remaining liveness question is whether the program makes progress. If progress itself composes for the facilities we use, then correctness can be reasoned about compositionally.

This is a position paper. It does not propose wording. Instead, it proposes a design principle for SG1: new C++ concurrency facilities should preserve safety, minimize undefined behavior, and expose progress guarantees strong enough for users to reason about correctness without re-proving the behavior of the whole program.

The observations in this paper may be familiar to many readers, but making them explicit gives us a useful lens for evaluating C++ concurrency facilities.

§ 1 Introduction

The C++ standard already contains a vocabulary for forward progress. We distinguish concurrent, parallel, and weakly parallel forward progress, and recent execution work exposes such guarantees through schedulers. These guarantees are not merely performance information. They determine which synchronization patterns are valid, which compositions may deadlock, and which abstractions can be used safely in generic concurrent code.

At the same time, much of the committee discussion around safety focuses on preventing bad states: memory safety, data-race freedom, arithmetic safety, lifetime safety, contract violations, and undefined behavior. These are essential, but they are not the whole of correctness. A program that never produces a wrong answer may still fail to produce any answer. For concurrent programs, that distinction is not theoretical; deadlocks, livelocks, starvation, and insufficient scheduling guarantees are ordinary ways in which otherwise safe components fail to form a correct program.

This paper uses Lamport’s safety/liveness distinction as a lens for C++ library design. The main claim is simple:

If safety holds, liveness reduces to progress. If progress composes, correctness composes.

The rest of the paper develops this claim. First, it recalls the safety/liveness distinction and explains why partial correctness is a safety property. Then it discusses why safety properties compose more naturally than liveness properties. The paper then pivots from liveness to progress and uses a task queue as an example of a facility whose guarantees can support compositional reasoning. Finally, it draws standardization takeaways: prefer facilities with clear progress guarantees, avoid facilities whose progress story cannot be made compositional, and treat undefined behavior as corrosive to both safety and progress.

Some of these ideas are also discussed in a broader form in [Teodorescu26]. This paper repeats and adapts them for the context of C++ standardization.

§ 1.1 Lamport’s safety and liveness

A *correct program* is a program that fully meets its requirements, whether explicit or implicit. Reasoning about correctness directly is hard. To make it easier, Lamport proposes focusing on two different kinds of properties: *safety* properties and *liveness* properties [Lamport77]. We follow his definitions:

To prove the correctness of a program, one must prove two essentially different types of properties about it, which we call *safety* and *liveness* properties. A **safety property** is one which states that something will *not* happen. For example, the partial correctness of a single process program is a safety property. It states that if the program is started with the correct input, then it cannot stop if it does not produce the correct output. A **liveness property** is one which states that something *must* happen. An example of a liveness property is the statement that a program will terminate if its input is correct.

In this paper, whenever we discuss programs, components, or facilities X , we mean the following:

- correctness of X : all safety and liveness properties are upheld in X (alternatively, all requirements are met)
- safety of X : all safety properties are upheld in X
- liveness of X : all liveness properties are upheld in X

Using Lamport’s taxonomy, it is important to note that **safety is part of correctness**. Safety is not opposed to correctness. Improving safety, if we can talk of such a thing, means improving correctness.

It is also important to notice, following Lamport, that partial correctness is a safety property. Partial correctness says that, if a program produces an answer, that answer is correct. It does not say that the program eventually produces an answer. That missing part is termination, or more generally progress. Thus:

$$\textit{Total correctness} = \textit{Partial correctness} + \textit{Termination}$$

At first, this may seem to clash with the statement that safety properties say that something bad does not happen. But partial correctness can be read negatively: *wrong answers are not produced*, and this follows the way we defined safety.

This also shows why the wording of a property is not enough to classify it. A safety property may be written in positive or negative terms. For a property P , we can often move between “ P must not happen” and “whenever we observe an answer, $\neg P$ holds”.

Turning our attention to liveness, when we say *something must happen*, we mean that *something must eventually happen*. The problematic part of liveness is not what that something looks like, but the guarantee that it eventually happens. As we shall see, this is closely related to progress.

In Lamport's framing, safety ensures that invariants associated with parts of the program are not violated, while liveness ensures that the program eventually enters certain states.

Note: In this paper, we assume that partial correctness for the program at hand can be defined through invariants. For example, programs that require human evaluation to determine partial correctness are not relevant for this discussion. We also focus on liveness properties that can be understood as eventually reaching an acceptable observable state; other liveness requirements need their own explicitly stated progress obligations.

§ 1.2 Safety composability

A safety property is violated when an invariant is violated (whether that invariant was actually checked in the code, or whether we can reason about it after the fact). This means that detecting safety failures can always be done in a finite prefix of the execution.

If a program reaches completion and no invariant has been violated, then the program is safe. If, on the other hand, we detect a safety violation in a module, then no matter what we do for the rest of the execution, the program remains unsafe.

This implies that **safety composes**. If A is executed with its preconditions met and its invariants are not violated, and if B is executed with its preconditions met and its invariants are not violated, then the execution of A followed by the execution of B cannot break any of these invariants. The required qualification is that the composition must preserve the assumptions of both components: in particular, A must not invalidate the preconditions or invariants required by B, and B must not invalidate invariants that still matter after A has executed.

By looking at invariants, we can say that safety is local, structural, and compositional.

Partial correctness is one important safety property: it rules out wrong answers.

§ 1.3 Liveness composability

Liveness behaves differently than safety: no finite prefix can prove violation of a liveness property.

No matter how long the program has failed to produce an answer, the execution may still be extended with an answer later. Liveness violations are therefore not conclusively detectable in finite time.

If the execution of A produces an answer, and if the execution of B produces an answer, it does not mean that putting A and B together will still produce an answer. The dining philosophers problem is the classic example for such violations [\[DiningPhilosophers\]](#). Livelocks are violations of liveness.

This means that, **generally, liveness does not compose**.

Liveness is global, behavioral, and often scheduler-dependent. Reasoning about liveness is harder, and gets harder as the system becomes more complex.

§ 2 A pivot on liveness

We usually think of liveness as completely separate from safety. This makes our reasoning harder. But we are not forced to do that. We can look at liveness only after looking at safety; that is, we can reason about safety-predicated liveness.

If our goal is correctness, and it should be, we need to evaluate both safety and liveness. We are free to choose the order in which we evaluate them. However, if we look at safety first, and we know that our program is safe, we can simplify the reasoning needed for liveness.

Here is a small sketch of this observation. Safety, which includes partial correctness, states that “if an answer is produced, then it will not be a wrong answer”. Liveness is about proving that “the right answer is produced”, which means that “an answer is produced, and the answer is right”. But, because we predicate liveness on safety, we already know that “if an answer is produced, then it is the right answer”. Thus, to prove liveness while safety holds, we only need to prove that “an answer is produced”. For more details, please see [\[Teodorescu26\]](#).

This reduction applies to the class of liveness properties considered in this paper: properties that require the program to eventually reach an acceptable observable state. For such properties, once safety has established that any observable answer is acceptable, the remaining liveness obligation is progress towards producing such an answer.

Thus, **if safety holds, liveness is reduced to progress**.

§ 3 Correctness composability

Reducing liveness to progress still does not make liveness compose.

In this paper, we say that **progress composes** when local progress guarantees survive the use of a facility. That is, if components satisfy the progress assumptions required by the facility, then the resulting composed component provides a progress guarantee that follows from the facility contract itself, rather than from a fresh global proof of the whole program.

The useful shift is the following: if we know that progress composes, then, under safety guarantees, the liveness obligations considered here also compose. We already know that safety composes, and therefore correctness composes.

Thus, **if safety holds and progress composes, then correctness composes.**

In systems composed of multiple parts where progress properties can be shown to compose, correctness can be established compositionally if: - each part is partially correct (safety); - each part makes progress when scheduled and executed (local termination); - a component capable of producing the final observable answer is eventually scheduled.

§ 3.1 An example

Let us take an example: suppose we are building a task queue. We aim for correctness. More precisely, we want to guarantee that if the caller code provides the right guarantees, adding a task queue to the system will maintain the correctness of the application.

Using the above results, we can establish correctness by having local safety guarantees and structural conditions for ensuring progress.

This is an idealized task queue example. Real queues have shutdown, cancellation, resource limits, priorities, failures, and implementation-defined scheduling behavior. Those details matter, but they are not the point of the example. The point is that the queue contract must make the relevant assumptions explicit.

The work queue relies on the following assumptions: - all work items satisfy the safety properties of the program; - safety properties are independent of execution order; - each work item, when executed, eventually completes; - at least one work item is capable of producing the final observable answer; - such a work item is eventually submitted to the queue; - the underlying scheduler eventually executes work made available by the queue.

Under these assumptions, a correctly implemented work queue ensures that: - the safety properties of the program are preserved during execution; - submitted work items are made available to the underlying scheduler; - no submitted work item is lost while the queue remains active.

The first two assumptions, together with the first work queue guarantee, establish safety for the program. The remaining assumptions, together with the queue guarantees that submitted work is made available and not lost, establish progress. Having both safety, which includes partial correctness, and progress, we can conclude that the program is correct.

The correctness of the program can be established without global reasoning, relying only on local safety guarantees and compositional progress.

§ 3.2 Discussion

§ 3.2.1 Undefined behavior

Undefined behavior leads to the program executing outside the abstract machine imposed by the language. In popular terms, the program can do anything.

In the presence of undefined behavior, no other safety property can be guaranteed. Progress guarantees also suffer.

§ 3.2.2 On the difficulty of composing safety

Safety composes, but not all safety properties compose the same way. The composition of memory safety or arithmetic safety, for example, is easy. If neither of the two parts has memory safety or arithmetic issues, then it follows immediately that putting these parts together will not create any memory or arithmetic safety issue.

However, if we want to compose partial correctness, the work needed to prove that is far more complicated. One needs to map the requirements of the program into invariants throughout the code, and then reason that none of those invariants are violated. The main difficulty here is this mapping between requirements and invariants. That mapping is usually not trivial, and reasoning about it is often very hard.

Imagine, for example, the requirements for a 3D game. Most likely, the requirements do not mention anything about linear algebra, but a lot of the code deals with linear algebra. Decomposing a problem into parts often means that requirements need to map to those smaller parts, and usually this mapping means changing or refining the requirements.

§ 3.2.3 On the difficulty of composing progress

Progress can often be made to compose by using a small number of well-understood execution structures. The number of techniques used for dealing with work items is limited in our domain. We do not need a new concurrency structure for each program we write.

For example, we know that low-level synchronization with semaphores is prone to progress issues. On the other hand, if the program only uses task queues, assuming that the task-producing logic provides progress guarantees, then we can prove progress.

This is why certain structures should be preferred over others: they provide progress guarantees (under certain assumptions).

§ 3.2.4 Working with uncertainty

In practice, we rarely have complete proofs that a component is partially correct. More often, we have engineering confidence: tests, review, static analysis, contracts, type-system checks, operational experience, and local reasoning all increase our confidence that a component preserves its invariants. Such evidence may fall short of a proof, but it still gives us a basis for reasoning under uncertainty.

Safety composability remains useful under this kind of uncertainty. If it is very likely that A has no safety violations, and it is very likely that B has no safety violations, then it is

also very likely that putting A and B together has no safety violations, provided the composition does not break the assumptions under which those confidence claims were made. The resulting confidence may be slightly lower, but the reasoning is still local: we combine confidence in the parts with confidence that the composition preserves their assumptions.

Progress guarantees play a similar role. They do not prove that arbitrary user code terminates, but they can prevent the facility itself from becoming the reason progress is lost. A useful standard facility should make these assumptions visible, so that users can reason locally about whether their code fits within them.

§ 4 Takeaways

Based on the results highlighted by this paper, we believe that a good software engineer should strive to:

- ensure safety for all the parts of a program;
- use only facilities for which progress composes.

As the C++ standard committee, we can support these efforts by: - striving to minimize undefined behavior (reduce the number of undefined behavior points we might have in a new library, make it harder to reach that state); - striving to provide safe facilities and discuss safety issues when proposing new facilities; - avoiding standardizing components that cannot have progress guarantees; such facilities may still be justified when the risks are clearly understood and the benefits clearly outweigh those risks.

These are small things that we can do to improve the safety, liveness and correctness of our C++ programs.

§ 5 References

[DiningPhilosophers] Dining philosophers problem.

https://en.wikipedia.org/wiki/Dining_philosophers_problem

[Lamport77] Leslie Lamport. 1977. Proving the Correctness of Multiprocess Programs.

<https://lamport.azurewebsites.net/pubs/proving.pdf>

[Teodorescu26] Lucian Radu Teodorescu. 2026-05-10. Safety-Refined Liveness.

<https://lucteo.ro/2026/05/10/safety-refined-liveness/>